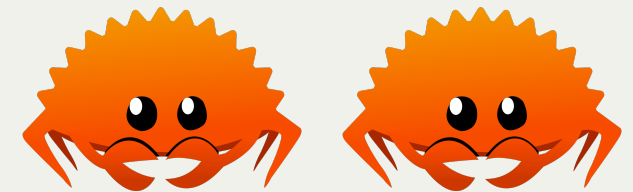


Семинар 3-4

- Времена жизни [Lifetimes]
- Умные указатели [Smart pointers]
- Функциональный стиль



Выделение памяти: стек [stack] и куча [heap]

Стек:

- Размещение данных последовательное (push/pop)
- Размер всех значений должен быть известен на момент компиляции
- Выделение памяти на стеке происходит быстрее

Куча:

- Память произвольного доступа, возможно выделить произвольный участок памяти
- В процессе выполнения можно запрашивать необходимый объём памяти для выделения
- Выделение памяти в куче медленнее, так как необходимо найти участок нужного размера

Где будет храниться значение? Стек или куча?

```
let a = 5;
```

```
let x = 5;  
let y = &x;
```

```
struct Point(i32, i32);  
let p = Point(3, 4);
```

```
let v = vec![1, 2];
```

Где будет храниться значение? Стек или куча?

-

```
let a = 5;
```

- Стек, но не всегда

-

```
let x = 5; let y = &x;
```

- Обе на стеке, но не всегда

-

```
struct Point(i32, i32); let p = Point(3, 4);
```

- Стек, но не всегда

-

```
let v = vec![1, 2];
```

- Сама структура хранится на стеке, а данные в куче, но не всегда

Использование Box<T>

```
1 let b = Box::new(5);  
2 println!("b = {b}");
```

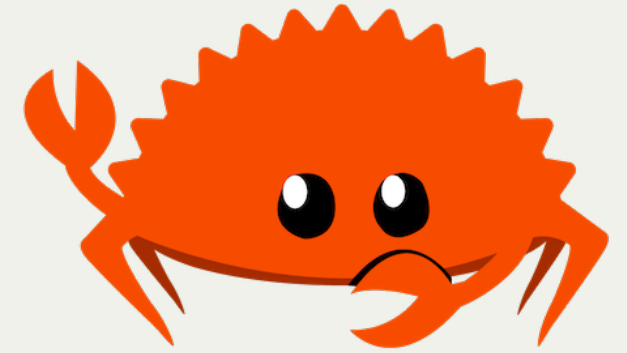
Когда использовать?

- Когда у вас есть тип, размер которого невозможно определить во время компиляции, и вы хотите использовать значение этого типа в контексте, требующем точного размера
- Когда у вас большой объём данных, и вы хотите передать право владения, но при этом гарантировать, что данные не будут скопированы
- Когда вы хотите владеть значением и вас интересует только то, реализует ли тип определённый трейт, а не то, принадлежит ли он к определённому типу

Пример рекурсивного типа

```
1 enum List {  
2     Cons(i32, Box<List>),  
3     Nil,  
4 }  
5  
6 use crate::List::{Cons, Nil};  
7  
8 fn main() {  
9     let list = Cons(1, Box::new(Cons(2, Box::new(Nil))));  
10 }
```

Время жизни [Lifetime]



Ограничения, накладываемые на ссылки

- Ссылка не может пережить своего референта
- Изменяемая ссылка не может иметь псевдоним [alias]
 - Переменные и указатели являются псевдонимами [aliases], если они ссылаются на перекрывающиеся области памяти
- Время жизни [lifetime token] представляет собой именованную часть кода, в которой ссылка должна быть валидной
- Исторически именовать можно было только лексические области видимости [lexical scope], что приводило к ряду случаев, когда валидный код отвергался компилятором
- Non-lexical lifetimes [(NLL)] в версии 1.63 изменили ситуацию
 - Анализируя граф потока управления программы, стало

возможным определять время жизни ссылки по ее фактическому
использованию

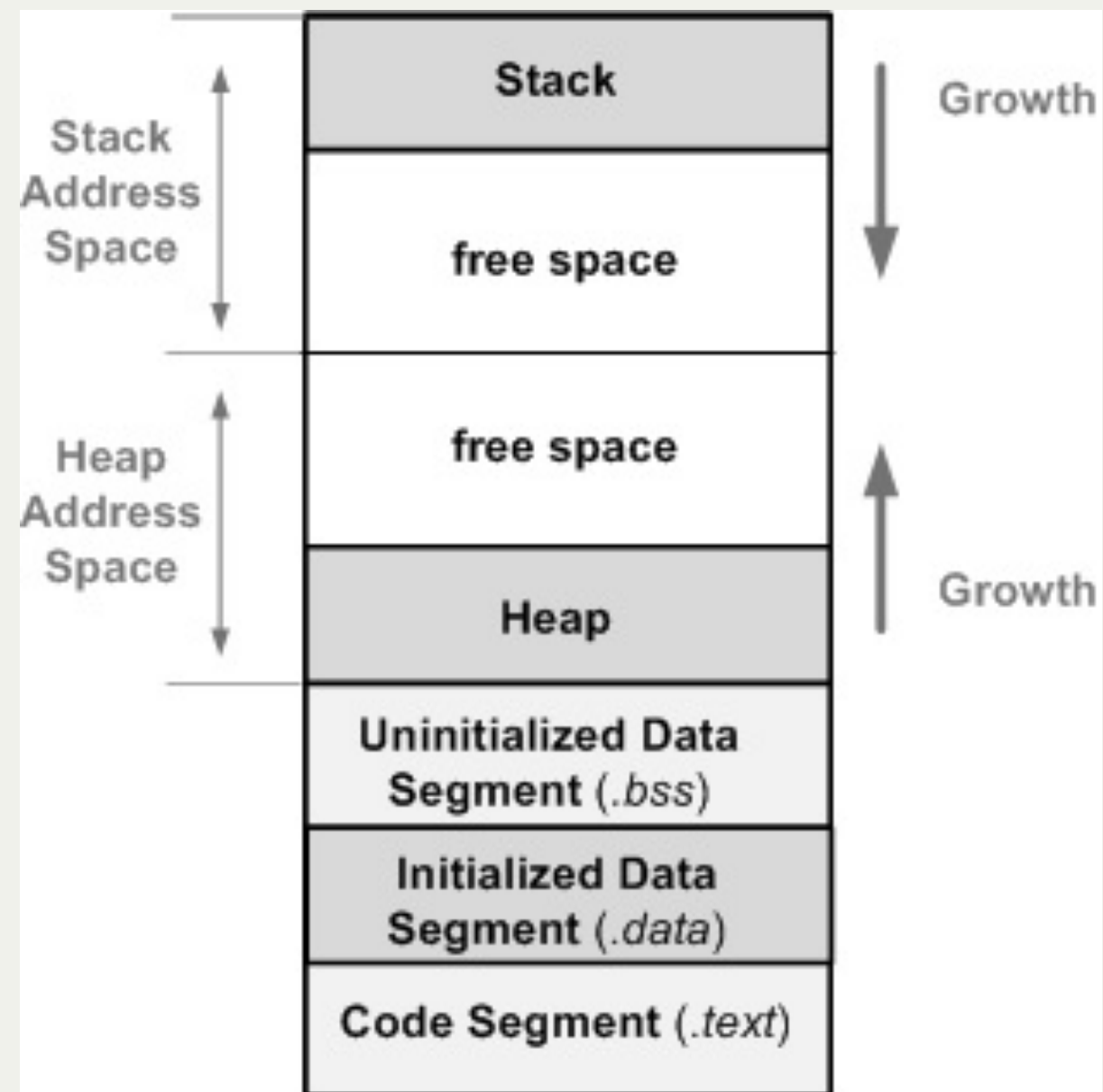
Static lifetime

- `'static` означает, что ссылка может существовать в течение всего срока выполнения программы
- Ключевые слова со специальными lifetime: `crate`, `Self`, `super`, `self`
- Стандартные примеры появления `'static` lifetime:
 - создание константы с помощью статического объявления
 - создание строкового литерала типа: `&'static str`

```
1 let s: &'static str = "I have a static lifetime.";
2 static NUM: i32 = 18;
```

Где храниться значения с `'static lifetime'`?

Где хранятся значения с 'static lifetime'?



Думай про NLL!



Пример lifetime в локальной области видимости

```
1 let x = 0;  
2 let y = &x;  
3 let z = &y;
```

```
1 // This lifetime syntax is invalid and is used as an example only.  
2 'a: {  
3     let x: i32 = 0;  
4     'b: {  
5         let y: &'b i32 = &'b x;  
6         'c: {  
7             let z: &'c &'b i32 = &'c y;  
8         }  
9     }  
10 }
```

Пример с висящей ссылкой

```
1 fn as_str(data: &u32) -> &str {  
2     let s = format!("{}", data);  
3     &s  
4 }
```

```
1 fn as_str<'a>(data: &'a u32) -> &'a str {  
2     'b: {  
3         let s = format!("{}", data);  
4         return &'a s  
5     }  
6 }
```

Пример псевдонима изменяемой ссылки

```
1 let mut data = vec![1, 2, 3];
2 let x = &data[0];
3 data.push(4);
4 println!("{}", x);
```

```
1 'a: {
2     let mut data: Vec<i32> = vec![1, 2, 3];
3     'b: {
4         let x: &'b i32 = Index::index::<'b>(&'b data, 0);
5         'c: {
6             Vec::push(&'c mut data, 4);
7         }
8         println!("{}", x);
9     }
10 }
```

Исправляем пример

```
1 let mut data = vec![1, 2, 3];
2 let x = &data[0];
3 println!("{}", x);
4 data.push(4);
```

```
1 'a: {
2     let mut data: Vec<i32> = vec![1, 2, 3];
3     'b: {
4         let x: &'b i32 = Index::index::<'b>(&'b data, 0);
5         println!("{}", x);
6     }
7     'c: {
8         Vec::push(&'c mut data, 4);
9     }
10 }
```

Опускание времени жизни [Elision]

- Где мы можем указывать время жизни?

```
1 &'a T
2 &'a mut T
3 T<'a>
```

- Примеры

```
1 fn print(s: &str); // elided
2 fn print<'a>(s: &'a str); // expanded
3 fn debug(lvl: usize, s: &str); // elided
4 fn debug<'a>(lvl: usize, s: &'a str); // expanded
5 fn get_str() -> &str; // ILLEGAL
6 fn frob(s: &str, t: &str) -> &str; // ILLEGAL
7 fn get_mut(&mut self) -> &mut T; // elided
8 fn get_mut<'a>(&'a mut self) -> &'a mut T; // expanded
```

Правила опускания времени жизни [Elision rules]

- Каждое опущенное время жизни на входе становится отдельным параметром
- Если есть только одно время жизни на входе (опущенное или нет), это время жизни присваивается *всем* опущенным временам жизни на выходе
- Если есть несколько времён жизни на входе, но одно из них `&self` или `&mut self`, время жизни `self` присваивается *всем* опущенным временам жизни на выходе
- Все другие случаи считаются ошибочными, если время жизни на выходе опущено



Пример функции с явным указанием lifetime

```
1 fn longest(x: &str, y: &str) -> &str {  
2     if x.len() > y.len() { x } else { y }  
3     // expected lifetime parameter  
4 }
```

```
1 fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {  
2     if x.len() > y.len() { x } else { y }  
3 }
```

```
struct ImportantExcerpt<'a> {  
    part: &'a str,  
}
```

Пример указания lifetime в структурах

```
1 struct Borrowed<'a>(&'a i32);
2 struct NamedBorrowed<'a> {
3     x: &'a i32,
4     y: &'a i32,
5 }
6 enum Either<'a> {
7     Num(i32),
8     Ref(&'a i32),
9 }
10 fn main() {
11     let x = 18; let y = 15;
12     let single = Borrowed(&x);
13     let double = NamedBorrowed { x: &x, y: &y };
14     let reference = Either::Ref(&x);
15     let number     = Either::Num(y);
16 }
```

RefCell<T>, внутренняя изменчивость

```
1 use std::cell::RefCell;
2 #[derive(Debug)]
3 struct MyStruct { value: RefCell<i32>, }
4 fn main() {
5     let my_instance = MyStruct {
6         value: RefCell::new(10),
7     };
8     println!("Initial value: {:?}", my_instance.value.borrow());
9     let mut_borrow = my_instance.value.borrow_mut();
10    *mut_borrow = 20;
11    println!("Value after: {:?}", my_instance.value.borrow());
12    let immutable_borrow = my_instance.value.borrow();
13    println!("Immutable borrow: {:?}", immutable_borrow);
14    // The following line would cause a runtime panic
15    let another_mut_borrow = my_instance.value.borrow_mut();
16    println!("Another mutable borrow: {:?}", another_mut_borrow);
17 }
```

Rc<T>, умный указатель с подсчетом ссылок

- Обратите внимание, что Rc<T> позволяет создавать циклические ссылки, которые могут привести к утечкам памяти

```
1 fn main() {
2     let a = Rc::new(Cons(5, Rc::new(Cons(10, Rc::new(Nil)))));
3     println!("count after creating a = {}", Rc::strong_count(&a));
4     let b = Cons(3, Rc::clone(&a));
5     println!("count after creating b = {}", Rc::strong_count(&a));
6     {
7         let c = Cons(4, Rc::clone(&a));
8         println!("count after creating c = {}",
9                 Rc::strong_count(&a));
10    }
11    println!("count after c goes out of scope = {}",
12            Rc::strong_count(&a));
13 }
```

Deref трейт

- Реализация Deref трейта позволяет использовать оператор * с структурой
- Под капотом будет вызвано `*(y.deref())`

```
1 use std::ops::Deref;
2 struct MyBox<T>(T);
3 impl<T> MyBox<T> {
4     fn new(x: T) -> MyBox<T> {
5         MyBox(x)
6     }
7 }
8 impl<T> Deref for MyBox<T> {
9     type Target = T;
10    fn deref(&self) -> &Self::Target {
11        &self.0
12    }
13 }
```

Drop трейт

```
1 struct CustomSmartPointer { data: String, }
2 impl Drop for CustomSmartPointer {
3     fn drop(&mut self) {
4         println!("Dropping CustomSmartPointer with data `{}`!",
5             self.data);
6     }
7 }
8 fn main() {
9     let c = CustomSmartPointer {
10         data: String::from("my stuff"),
11     };
12     let d = CustomSmartPointer {
13         data: String::from("other stuff"),
14     };
15     println!("CustomSmartPointers created.");
16 }
```

Замыкания [Closures] (1)

- Примеры аннотации замыканий

```
1 fn add_one_v1    (x: u32) -> u32 { x + 1 }
2 let add_one_v2 = |x: u32| -> u32 { x + 1 };
3 let add_one_v3 = |x|           { x + 1 };
4 let add_one_v4 = |x|           x + 1 ;
```

- Типизация замыкания

```
1 let example_closure = |x| x;
2 let s = example_closure(String::from("hello"));
3 // The following line will cause a compilation error
4 let n = example_closure(5);
```

Замыкания [Closures] (2)

- Замыкания могут захватывать переменные из своего окружения тремя способами:
 - неизменяемое заимствование [borrowing immutably]
 - изменяемое заимствование [borrowing mutably]
 - получение владения [taking ownership]

```
1 let list = vec![1, 2, 3];  
2 let only_borrows = || println!("From closure: {list:?}");  
3 let mut borrows_mutably = || list.push(7);
```

- Ключевое слово **move** позволяет замыканию получить владение захваченной переменной, даже если тело замыкания этого не требует

```
1 let list = vec![1, 2, 3];  
2 thread::spawn(move || println!("From thread: {list:?}"));
```

Трейты замыканий

- `FnOnce` трейт обозначает, что замыкание может быть вызвано хотябы один раз. Все замыкания реализуют как минимум этот трейт. Замыкание, которое перемещает захваченные значения из своего тела, будет реализовывать только `FnOnce` и ни один из других трейтов `Fn`.
- `FnMut` применяется к замыканиям, которые не перемещают захваченные значения из своего тела, но могут изменять захваченные значения.
- `Fn` применяется к замыканиям, которые не перемещают захваченные значения из своего тела и не изменяют захваченные значения, а также к замыканиям, которые ничего не захватывают из своего окружения.



Пример использования трейтов замыканий

```
1 impl<T> Option<T> {
2     pub fn unwrap_or_else<F>(self, f: F) -> T
3     where
4         F: FnOnce() -> T
5     {
6         match self {
7             Some(x) => x,
8             None => f(),
9         }
10    }
11 }
```

Пример использования трейтов замыканий

```
1 fn apply<F>(f: F) where
2     F: FnOnce() {
3     f();
4 }
5 fn create_fn() -> impl Fn() {
6     let text = "Fn".to_owned();
7     move || println!("This is a: {}", text)
8 }
9 fn create_fnmut() -> impl FnMut() {
10     let text = "FnMut".to_owned();
11     move || println!("This is a: {}", text)
12 }
13 fn create_fnonce() -> impl FnOnce() {
14     let text = "FnOnce".to_owned();
15     move || println!("This is a: {}", text)
16 }
```

Итераторы [Iterators]

- Итераторы в Rust ленивые
- Все итераторы реализуют трейт `Iterator`

```
1 pub trait Iterator {  
2     type Item;  
3     fn next(&mut self) -> Option<Self::Item>;  
4     // methods with default implementations elided  
5 }
```

- Пример использования адаптеров [iterator adapter]

```
1 let v1: Vec<i32> = vec![1, 2, 3];  
2 let v2: Vec<_> = v1.iter().map(|x| x + 1).collect();  
3 assert_eq!(v2, vec![2, 3, 4]);
```

Ассоциированные типы [Associated types]

- Ассоциированный тип связывает тип с трейтом, позволяя использовать ассоциированный тип в своих сигнатурах

```
1 impl Iterator for Counter {  
2     type Item = u32;  
3     fn next(&mut self) -> Option<Self::Item> {  
4         // --snip--  
    }
```

- Отличается от обобщений тем, что не позволяет делать множество реализаций для разных типов

```
1 pub trait Iterator<T> {  
2     fn next(&mut self) -> Option<T>;  
3 }
```

Юнит тестирование

```
1 pub fn foo(a: u64, b: u64) -> u64 {
2     if b == 0 { panic!("b can't be 0") }
3     a / b
4 }
5 #[cfg(test)]
6 mod tests {
7     use super::*;
8     #[test]
9     fn simple() {
10         let result = foo(4, 2);
11         assert_eq!(result, 2);
12     }
13     #[test]
14     #[should_panic]
15     fn devide_by_zero() { foo(4, 0); }
16 }
```

Практика

- Разработайте структуру данных, представляющую ориентированный граф без кратных дуг на языке Rust
- Задачи:
 - представить граф в памяти с помощью списков/массивов смежности
 - реализовать абстрактный тип данных (АТД) граф
 - реализовать итератор для обхода графа в глубину, используя трейт `Iterator`
 - написать юнит тесты к своей реализации

